Specificațiile limbajului

Aspecte generale

Am definit o semantică operațională big-step pentru un limbaj de programare ce permite lucrul cu funcții, blocuri de cod, declarații/asignări de variabile, structuri decisive, structuri repetitive, liste cu valori numerice. Limbajul de programare e capabil să detecteze erori și să ofere mesaje de atenționare detaliate legate de erori.

Sintaxa

AExp - Expresiile aritmetice acceptate

AExp ::= n (valoare numerică)

| x (variabilă)

| AExp +' AExp (adunări)

| AExp -' AExp (scăderi)

| AExp *' AExp (înmulțiri)

| AExp /' AExp (împărțiri)

| AExp %' AExp (restul împărțirii)

| AExp.head(LExp) (primul element al unei liste)

| AExp.length(LExp) (lungimea unei liste)

| AExp.nth(LExp, AExp) (al n-lea element al unei liste)

LExp - Expresii listă

LExp ::= LExp.empty Tip.Int (listă goală de un tip, în principiu Int; sistemul de tipuri permite si alte tipuri la declarare de liste goale, dar liste cu alte tipuri nu pot fi construite)

| LExp.cons(AExp, LExp) (adaugă un element de tip AExp la începutul unei liste)

| x (variabila listă)

| LExp.append(LExp, LExp) (concatenarea a două liste, ordinea din rezultatul e identică cu ordinea parametrilor)

BExp - Expresii boolene

BExp ::= BExp.T(adevărat)

| BExp.F (fals)

| x (variabila de tip bool)

| AExp ==' AExp (egalitate)

| AExp <' AExp (mai mic)

| AExp <=' AExp (mai mic egal)

| !' BExp (negare)

| BExp and' BExp (conjunctie)

| BExp or` BExp (disjunctie)
| isEmpty(LExp) (verificare lista goala)

Com - comenzi (statement-uri)
Com ::= skip
| x ::= AExp (asignare numerica)
| x ::=b BExp (asignare valoare logica)
| x ::=l LExp (asignare lista)
| x :::= Tip.Int, AExp (declarare variabila numar)
| x :::=b BExp (declarare variabila booleana)
| x :::=l LExp (declarare lista)
| Com ;; Com (secventa de cod)
| if BExp then Com else Com (structura decizionala)
| while BExp do Com (structura repetitiva)
| for (Com; BExp; Com) Com (structura repetitiva)
| { Com } (block scope)
| declaratie_functie (nume) [(nume(i),Tip(i))] Tip Com (declaratia de functie)
| apel_functie (nume_functie_declarata) [argumente] (apel functie ce returneaza Int)
| apel_functie_bool (nume_functie_declarata) [argumente] (apel functie ce returneaza bool)
| return x (specific functiilor, adauga pe stiva rezultatelor o valoare)
| pop x (ia varful stivei, baga valoarea intr-o variabila si actualizeaza stiva)

<u>Definiția configurației</u>

Executarea unei secvente de cod va schimba configuratia in care se afla programul la un moment dat. O configurație EvalState este un tuplu ⟨c,σ,D,FE,S⟩, unde :

- c = bucata de cod ce urmează a fi executată
- env = String ->Value. Environment-ul pentru variabile; va returna pentru fiecare variabilă valoarea ei din domeniul de vizibilitate al lui env. Este de tipul Env.
- decl = o funcție String->Bool care returnează true dacă o variabilă a fost declarată în domeniul de vizibilitate actual sau fals altfel; este necesară în gestionarea domeniilor de vizibilitate. Este de tipul Decl.
- tipenv = String -> Option Tip. Returneaza tipul variabilei cu nume indicat de String sau none daca nu a fost declarata.
- funenv = function environment; este o functie String -> Option (List (String × Tip) × Tip × Com) care reține funcțiile declarate. Ea asociază fiecărui nume de funcție perechea corespunzătoare formată din lista de parametri și din secvența

de cod executată de funcție. Pentru fiecare parametru este retinut atat numele cat si tipul cu care este utilizat el in codul functiei. Este de tipul FunEnv

- stack = List Value. Stiva rezultatelor de funcții va reține rezultatele calculate de funcții. Este de tipul Stack.

Sistemul de tipuri

Limbajul are un sistem de tipuri static asigură că participanții la o operație sau parametri unei funcții din limbajul de programare au tipurile corecte. Această verificare este realizată înainte de executarea propriu-zisă a codului. Concret, operațiile aritmetice trebuie realizate doar între lucruri ce pot fi evaluate la valori numerice, operațiile logice doar între lucruri ce au o valoare de adevăr ce poate fi dedusă, iar operațiile între liste trebuie permise doar între liste. Pentru a realiza toate acestea, cum se observă de altfel și din sintaxa, e necesară menționarea tipului variabilelor la declararea acestora.

- Tip poate fi Int, Bool sau List
- Value poate fi intVal, boolVal sau listVal
- tipenv verifică dacă o variabilă a fost declarată anterior și dacă e folosită corespunzător relativ la tipul respectiv

Pe baza acestor 3 elemente și a regulilor din typeCheckAExp, typeCheckBExp, typeCheckLExp și eval limbajul detectează erori și oferă informații legate de acestea.

Erori detectate de sistemul de tipuri

- UndeclaredVariable/UndeclaredFunction - se utilizează un nume de variabilă/funcție, dar nu există vreo variabilă declarată cu acel nume
- TypeMismatch - se folosesc operații sau funcții pe termeni care au tip necorespunzător. De exemplu, adunarea între valorile boolene.
- VariableAlreadyDeclared/FunctionAlreadyDeclared - se încearcă redeclararea unei variabile (cu toate că, dacă scope-ul este schimbat, poți redeclara o variabilă pe acel scope - va fi o secțiune separată cu asta)
- NoCondition - comenzile de tipul while, if, for au nevoie de o condiție logică, de tip Bool, ca să funcționeze
- WrongType - declarări/asignări la variabile cu tipuri necorespunzătoare are expresiilor folosite. De exemplu, asignarea unei valori de tip LExp la o variabilă de tip Int.

Alte erori detectate

Există și alte erori care sunt detectate, dar ele sunt detectate la runtime, nu static.

- DivisionByZero - încerci să împarți la zero
- EmptyList - încerci să extragi primul element al unei liste care nu are niciun element
- IndexOutofBounds - încerci să accesezi un index din afara unei liste

<u>Domenii de vizibilitate</u>

Semantica va face distincția între

```
"x" :::= 1 ;;
 {
   "x" :::= 2
 }
```

și

```
"x" :::= 1;;
 {
   "x" ::= 2
 }
```

În primul caz, x va fi în final 1, în cel de-al doilea va fi 2. Modificările din blocurile de cod vor fi păstrate doar pentru acele variabile care nu au fost declarate în blocul respectiv. (un exemplu si mai elocvent este ex_block_scope din interpretor). De asemenea, declarațiile de funcții nu vor modifica valorile variabilelor, ci vor modifica doar function environment-ul. Asemănător, apelul de funcții va modifica doar stiva de rezultate.

<u>Semantica</u> (se presupune ca toate instructiunile sunt realizate cu succes, fara erori)

Evaluarea Big-Step

- $\langle c,\sigma,D,F,S\rangle \Downarrow \langle \sigma',D',F',S'\rangle$

Semantica AExp

num :

$$\frac{}{\langle \text{num } i\ ,\ \sigma\rangle \Downarrow \langle i\rangle}$$

var :

$\langle var\ x\ ,\ \sigma\rangle \Downarrow \langle \sigma(x)\rangle$

adunare :

$\langle a1\ ,\ \sigma\rangle \Downarrow \langle v1\rangle \qquad \langle a2\ ,\ \sigma\rangle \Downarrow \langle v2\rangle$

$\langle a1\ +'\ a2\ ,\ \sigma\rangle \Downarrow \langle v1 + v2\rangle$

scadere :

$\langle a1\ ,\ \sigma\rangle \Downarrow \langle v1\rangle \qquad \langle a2\ ,\ \sigma\rangle \Downarrow \langle v2\rangle$

$\langle a1\ -'\ a2\ ,\ \sigma\rangle \Downarrow \langle v1 - v2\rangle$

inmultire :

$\langle a1\ ,\ \sigma\rangle \Downarrow \langle v1\rangle \qquad \langle a2\ ,\ \sigma\rangle \Downarrow \langle v2\rangle$

$\langle a1\ *'\ a2\ ,\ \sigma\rangle \Downarrow \langle v1 * v2\rangle$

impartire :

$\langle a1\ ,\ \sigma\rangle \Downarrow \langle v1\rangle \qquad \langle a2\ ,\ \sigma\rangle \Downarrow \langle v2\rangle$

$\langle a1\ /'\ a2\ ,\ \sigma\rangle \Downarrow \langle v1 / v2\rangle$

modulo :

$\langle a1\ ,\ \sigma\rangle \Downarrow \langle v1\rangle \qquad \langle a2\ ,\ \sigma\rangle \Downarrow \langle v2\rangle$

$\langle a1\ \%'\ a2\ ,\ \sigma\rangle \Downarrow \langle v1\ \%\ v2\rangle$

AExp.head :

$\langle\ l\ ,\ \sigma\ \rangle \Downarrow \langle\ h :: t\ \rangle$

$$\frac{}{\langle \text{AExp.head } l , \sigma \rangle \Downarrow \langle h \rangle}$$

AExp.length :

$$\frac{\langle l , \sigma \rangle \Downarrow \langle L \rangle}{\langle \text{AExp.length } l , \sigma \rangle \Downarrow \langle |L| \rangle}$$

AExp.nth :

$$\frac{\langle l , \sigma \rangle \Downarrow \langle L \rangle \quad \langle i , \sigma \rangle \Downarrow \langle n \rangle \quad 0 \leq n < |L|}{\langle \text{AExp.nth } l\ i , \sigma \rangle \Downarrow \langle L[n] \rangle}$$

Semantica BExp

T :

$$\frac{}{\langle \text{BExp.T} , \sigma \rangle \Downarrow \langle \text{true} \rangle}$$

F :

$$\frac{}{\langle \text{BExp.F} , \sigma \rangle \Downarrow \langle \text{false} \rangle}$$

mai_mic :

$$\frac{\langle a1 , \sigma \rangle \Downarrow \langle v1 \rangle \qquad \langle a2 , \sigma \rangle \Downarrow \langle v2 \rangle}{\langle a1 <' a2 , \sigma \rangle \Downarrow \langle v1 < v2 \rangle}$$

mai_mic_sau_egal :

$$\frac{\langle a1 , \sigma\rangle \Downarrow \langle v1\rangle \qquad \langle a2 , \sigma\rangle \Downarrow \langle v2\rangle}{\langle a1 \text{ <=' } a2 , \sigma\rangle \Downarrow \langle v1 \leq v2\rangle}$$

negatie :

$$\frac{\langle b , \sigma\rangle \Downarrow \langle true\rangle}{\langle \text{!' } b , \sigma\rangle \Downarrow \langle false\rangle}$$

$$\frac{\langle b , \sigma\rangle \Downarrow \langle false\rangle}{\langle \text{!' } b , \sigma\rangle \Downarrow \langle true\rangle}$$

conjunctie :

$$\frac{\langle b1 , \sigma\rangle \Downarrow \langle true\rangle \quad \langle b2 , \sigma\rangle \Downarrow \langle v\rangle}{\langle b1 \text{ and' } b2 , \sigma\rangle \Downarrow \langle v\rangle}$$

$$\frac{\langle b1 , \sigma\rangle \Downarrow \langle false\rangle}{\langle b1 \text{ and' } b2 , \sigma\rangle \Downarrow \langle false\rangle}$$

egalitate :

$$\frac{\langle a1 , \sigma\rangle \Downarrow \langle v1\rangle \langle a2 , \sigma\rangle \Downarrow \langle v2\rangle}{\langle a1 \text{ ==' } a2 , \sigma\rangle \Downarrow \langle v1 = v2\rangle}$$

disjunctie :

$$\frac{\langle b1 , \sigma\rangle \Downarrow \langle true\rangle}{}$$

$\langle b1 \text{ or' } b2 , \sigma\rangle \Downarrow \langle true\rangle$

$$\frac{\langle b1 , \sigma\rangle \Downarrow \langle false\rangle \qquad \langle b2 , \sigma\rangle \Downarrow \langle v\rangle}{\langle b1 \text{ or' } b2 , \sigma\rangle \Downarrow \langle v\rangle}$$

isEmpty :

$$\frac{\langle l , \sigma \rangle \Downarrow \langle [] \rangle}{\langle \text{ isEmpty } l , \sigma \rangle \Downarrow \langle \text{ true } \rangle}$$

$$\frac{\langle l , \sigma \rangle \Downarrow \langle h :: t \rangle}{\langle \text{ isEmpty } l , \sigma \rangle \Downarrow \langle \text{ false } \rangle}$$

Semantica LExp

empty :

$$\frac{}{\langle \text{ empty } t , \sigma \rangle \Downarrow \langle [] \rangle}$$

cons :

$$\frac{\langle a , \sigma \rangle \Downarrow \langle v \rangle \qquad \langle l , \sigma \rangle \Downarrow \langle L \rangle}{\langle \text{ cons } a\ l , \sigma \rangle \Downarrow \langle v :: L \rangle}$$

var :

$$\frac{\sigma(x) = \text{listVal } L}{\langle \text{ var } x , \sigma \rangle \Downarrow \langle L \rangle}$$

append :

$$\frac{\langle l1 , \sigma \rangle \Downarrow \langle L1 \rangle \quad \langle l2 , \sigma \rangle \Downarrow \langle L2 \rangle}{\langle \text{append } l1\ l2 , \sigma \rangle \Downarrow \langle L1 \text{ ++ } L2 \rangle}$$

Semantica Com

skip :

$$\frac{}{\langle \text{skip} , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma, D, T, F, S \rangle}$$

atribuire :

$$\frac{\langle a , \sigma \rangle \Downarrow \langle v \rangle}{\langle x ::= a , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma[x \mapsto v], D, T, F, S \rangle}$$

atribuire_bool :

$$\frac{\langle b , \sigma \rangle \Downarrow \langle v \rangle}{\langle x ::=b\ b , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma[x \mapsto v], D, T, F, S \rangle}$$

atribuire_list :

$$\frac{\langle l , \sigma \rangle \Downarrow \langle L \rangle}{\langle x ::=l\ l , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma[x \mapsto L], D, T, F, S \rangle}$$

declaratie :

$$\frac{\langle a , \sigma \rangle \Downarrow \langle v \rangle}{\langle x :::= a , \sigma, D, T, FE, S \rangle \Downarrow \sigma[x \mapsto v] , \text{updateDecl } D\ x, \text{updateTip } T\ x\ \text{Int} ,FE, S \rangle}$$

declaratie_bool :

$$\frac{\langle b , \sigma \rangle \Downarrow \langle v \rangle}{\langle x ::= b , \sigma, D, T, FE, S\rangle \Downarrow \langle \sigma[x \mapsto v] , updateDecl\ D\ x, updateTip\ T\ x\ Bool\ FE, S\rangle}$$

declaratie_list :

$$\frac{\langle l , \sigma \rangle \Downarrow \langle L \rangle}{\langle x ::= l , \sigma, D, T, FE, S\rangle \Downarrow \langle \sigma[x \mapsto L] , updateDecl\ D\ x, updateTip\ T\ x\ Tip.List\ \ FE, S\rangle}$$

secventa :

$$\frac{\langle c1 , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle \quad \langle c2 , \sigma', D', T', F', S' \rangle \Downarrow \langle \sigma'', D'', T'', F'', S'' \rangle}{\langle c1 ;; c2 , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma'', D'', T'', F'', S'' \rangle}$$

ite :

$$\frac{\langle b , \sigma \rangle \Downarrow \langle true \rangle \quad \langle c1 , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle}{\langle ite\ b\ c1\ c2 , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle}$$

$$\frac{\langle b , \sigma \rangle \Downarrow \langle false \rangle \quad \langle c2 , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle}{\langle ite\ b\ c1\ c2 , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle}$$

while :

$$\frac{\langle b , \sigma \rangle \Downarrow \langle false \rangle}{\langle while\ b\ c , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma, D, T, F, S \rangle}$$

$\langle b , \sigma \rangle \Downarrow \langle true \rangle$

$\langle c , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle$

$\langle while\ b\ c , \sigma', D', T', F', S' \rangle \Downarrow \langle \sigma'', D'', T'', F'', S'' \rangle$

$\langle while\ b\ c , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma'', D'', T'', F'', S'' \rangle$

for :

$\langle init , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma'', D'', T'', F'', S'' \rangle$

$\langle while\ cond\ (body\ ;;\ pas) , \sigma'', D'', T'', F'', S'' \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle$

$\langle for\ init\ cond\ pas\ body , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle$

bloc :

$\langle c , \sigma, \varnothing, T, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle$

$\langle \{ c \} , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma \triangleright \sigma', D, T \triangleright T', F, S' \rangle$

declaratie_functie :

$\langle declaratie_functie\ f\ ps\ t\ c , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma, D, T, F[f \rightarrow (ps,t,c)], S \rangle$

apel_functie :

$\langle args , \sigma \rangle \Downarrow \langle vs \rangle$

$\langle body , \sigma p, \varnothing, Tp, F, S \rangle \Downarrow \langle \sigma', D', T', F', S' \rangle$

$\langle apel_functie\ f\ args , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma, D, T, F, S' \rangle$

return :

$\langle return\ x , \sigma, D, T, F, S \rangle \Downarrow \langle \sigma, D, T, F, (\sigma(x) :: S) \rangle$

pop :

$S = v :: S'$

$\langle \text{pop } x, \sigma, D, T, F, S \rangle \Downarrow \langle \sigma[x \mapsto v], D, T, F, S' \rangle$

<u>Decizii legate de design</u>

- Separarea value environment si type environment permite verificarea statica a tipurilor inainte de executia propriu-zisa a codului
- eval (fuel : Nat) (c : Com) (s : EvalState) : Except EvalError EvalState - folosirea „Except EvalError” permite surprinderea si propagarea erorilor detectate
- fuel-ul din eval e necesar pentru a garanta terminarea (lucru necesar in lean4)
- rolul Repr instance - face afisarea rezultatelor determinate de eval mult mai usor de citit

<u>Funcții auxiliare</u>

Gestionarea environmentului de variable

- update (sigma : Env) (var : String) (val : Int) : Env - returnează un nou environment unde variabila var are noua valoare val, iar celelalte evaluări rămân la fel

Gestionarea mecanismului domeniilor de vizibilitate

- updateDecl (D : Decl) (x : String) : Decl - returnează o nouă funcție D unde variabila x va fi acum evaluată la true, iar celelalte evaluări se fac la fel
- updateBloc (sigma_initial : Env) (sigma_final : Env) (scopeDecl : Decl) : Env - returnează un nou environment, folosită la blocuri de cod; atunci când terminăm de executat un bloc de cod vrem să păstrăm modificările făcute doar pe acele variabile care nu au fost declarate în acel bloc de cod. Astfel, cele redeclarate vor avea scopeDecl setat la true și se vor evalua la valoarea lor dinaintea execuției blocului de cod (sigma_initial). Celelalte vor avea scopeDecl setat la false și se vor evalua la valoarea lor finală din bloc (sigma_final).

Gestionarea functiilor

- updateFun (F : FunEnv) (name : String) (parametri : List String) (cod : Com) : FunEnv - returnează un nou function environment unde funcția name va fi evaluată la perechea (parametri, cod), iar celelalte evaluări rămân la fel

- stack_push (v : Int) (S : Stack) : Stack - returnează un nou stack care păstrează celelalte valori dar care are valoarea v în vârf
- stack_pop (S : Stack) : (Int x Stack) - returnează perechea formată din elementul din vârful stivei S și stiva S fără acel element
- update_params (Env)(List String)(List Int) : Env - returnează un nou environment în care valorile parametrilor din funcție sunt egale cu valorile date în apelul funcției

Gestionarea Type Environment

- updateTip (T : TipEnv) (var : String) (tip : Tip) : TipEnv - returnează un nou type environment unde variabila var va fi asociată cu tipul tip, iar celelalte asocieri rămân neschimbate. Această funcție este esențială pentru verificarea statică a tipurilor în timpul execuției.
- update_params_tip (tipenv : TipEnv) (params : List (String × Tip)) : TipEnv - returnează un nou type environment în care parametrii funcției primesc tipurile lor corespunzătoare. Se aplică recursiv: pentru fiecare pereche (parametru, tip) din listă, se actualizează type environment-ul folosind updateTip.
- updateBlocTip (tipenv_initial : TipEnv) (tipenv_final : TipEnv) (scopeDecl : Decl) : TipEnv - analog cu updateBloc, dar pentru type environment. Returnează un nou type environment care păstrează doar modificările făcute pe variabilele care nu au fost declarate în blocul curent. Variabilele redeclarate (pentru care scopeDecl este true) revin la tipurile lor inițiale.

Gestionarea stivei și evaluarea argumentelor pentru functii

- eval_args (args : List AExp) (sigma : Env) : Except EvalError (List Int) - evaluează o listă de expresii aritmetice în contextul environment-ului sigma și returnează o listă de valori întregi. Această funcție este utilizată pentru evaluarea argumentelor înainte de apelul unei funcții. În caz de eroare la evaluarea oricărui argument, propagă eroarea.
- update_params (sigma : Env) (params : List (String × Tip)) (vals : List Int) : Env - versiune extinsă care primește perechile (nume_parametru, tip_parametru) și le asociază cu valorile corespunzătoare din vals. Se aplică recursiv până când ambele liste sunt epuizate.

Funcția de afișare și debugging

- showDeclaredVarsWithTypes (sigma : Env) (decl : Decl) (tipenv : TipEnv) : String - generează o reprezentare text a tuturor variabilelor declarate împreună cu tipurile și valorile lor curente. Filtrează doar variabilele care sunt efectiv declarate (pentru care decl returnează true) și formatează output-ul în funcție de tip:
    - Pentru Tip.Int: afișează nume: Int = valoare
    - Pentru Tip.Bool: afișează nume: Bool = true/false
    - Pentru Tip.List: afișează nume: List = [...]